LLM GATEWAY

Roll out an LLM gateway for your organization

Deploy a gateway product for Claude Code: configure it to forward what Claude Code sends, issue developer credentials, distribute the configuration through managed settings, and verify the rollout.

This page walks an administrator through rolling out an LLM gateway for Claude Code. It assumes you have a gateway product deployed that meets the **gateway requirements**. Deploying or operating any specific product isn't covered here; deploy yours following its vendor's documentation.

- To connect Claude Code on your own machine to an existing gateway, see **Connect Claude Code to an LLM gateway**
- For what Claude Code sends to a gateway and what to forward, see the **gateway protocol reference**

Prerequisites

To complete the rollout, you'll need:

- A gateway deployed on your infrastructure, serving HTTPS at the exact address you'll distribute to developers, not an address that redirects to it, and configured to route Claude model names to your provider
- A provider credential for the gateway to forward with:
  - For the Anthropic API: an API key from the **Claude Console**
  - For a cloud provider: cloud credentials with model access. See the prerequisites on the **Amazon Bedrock**, **Google Vertex AI**, or **Microsoft Foundry** page
- A way to deliver settings files to developer machines, such as MDM or configuration management
  - If you don't have one yet, **how settings reach devices** compares the options

Gateway requirements

Whichever product provides the gateway, it must:

- **Accept a supported API format**: one of the formats in the **API formats table**. The rollout steps below assume the Anthropic Messages API at `POST /v1/messages`, which most gateways serve
- **Stream responses**: pass server-sent events through as they arrive instead of buffering the whole response
- **Route Claude model names**: map each name developers use to an upstream model. Claude Code sends a model name such as `claude-sonnet-4-6` in each request; in most gateway products the mapping is a model list or routing table in the gateway's own configuration
- **Forward headers and body unchanged**: pass `anthropic-beta`, `anthropic-version`, and the request body through in both directions; the **feature pass-through table** maps each to the feature that breaks without it
- **Return upstream errors unmodified**: Claude Code's automatic recovery matches on error wording, so wrapping errors in the gateway's own envelope breaks it
- **Exempt the path from request-body WAF inspection**: Claude Code prompts carry source code and XML-style tags that match cross-site-scripting body rules; a WAF in front of the gateway returns `403` on real sessions while short test requests pass

Optionally, serve `GET /v1/models` so Claude Code can populate the model picker from your gateway with **model discovery**.

Rollout steps

The rollout takes five steps, each with a checkpoint:

1. **Confirm the gateway routes your models**
2. **Issue each developer a credential**
3. **Test Claude Code against the gateway**
4. **Distribute the base URL and credentials**
5. **Verify from a developer machine**

The steps involve three different credentials, and the checkpoints name them by placeholder so you can tell which one is at fault when something fails:

Credential	Who holds it	Placeholder in checkpoints
Provider credential	The gateway, which forwards it to the upstream provider	Configured on the gateway; never appears in client commands
Gateway administrative credential	You, if your gateway product issues one for its admin or test interface	`<gateway-key>`
Developer key	Each developer, issued by the gateway in **Issue developer credentials**	`<developer-key>`

Confirm the gateway routes your models

Your gateway should already be configured with your provider credential, listening at its base URL, and forwarding requests to your provider's API. Test that the path works end to end with a minimal request, substituting two values from your deployment:

- `<gateway-key>` is whatever credential lets you call the gateway right now: an administrative key, a test key, or your own developer key if you've already issued one. Not every gateway product has a separate admin credential; if yours doesn't, issue yourself a developer key in **Issue developer credentials** first
- `model` is a Claude model name your gateway is configured to route. The example uses `claude-sonnet-4-6`; substitute a name you've configured

Bash or Zsh

```
curl -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <gateway-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

PowerShell

```powershell
Invoke-RestMethod -Method Post -Uri "https://llm-
gateway.example.com/v1/messages" `
  -Headers @{ "Authorization" = "Bearer <gateway-key>";
"anthropic-version" = "2023-06-01" } `
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```bash
curl -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <developer-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```powershell
Invoke-RestMethod -Method Post -Uri "https://llm-
gateway.example.com/v1/messages" `
  -Headers @{ "Authorization" = "Bearer <developer-key>";
"anthropic-version" = "2023-06-01" } `
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```bash
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN="<developer-key>"
```

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "<developer-key>"
```

```
curl -N -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <developer-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 16, "stream":
true, "messages": [{"role": "user", "content": "count to 3"}]}'
```

```
$body = '{"model": "claude-sonnet-4-6", "max_tokens": 16,
"stream": true, "messages": [{"role": "user", "content": "count
to 3"}]}'
$body | curl.exe -N -X POST "https://llm-
gateway.example.com/v1/messages" `
  -H "Authorization: Bearer <developer-key>" `
  -H "anthropic-version: 2023-06-01" `
  -H "content-type: application/json" `
  --data-binary '@-'
```

Checkpoint: a `200` with a `content` field means the gateway reached the provider with that model name. A `404` means that name isn't routed at the gateway; a `401` from the provider means the gateway's provider credential is wrong.

Repeat the request once per Claude model name in your gateway's routing configuration. A name the gateway doesn't route returns `404` to any developer who selects it, so test every name before rollout.

> Avoid serving the gateway behind a redirect. A redirect can drop the request body or strip the credential header on inference requests, and **model discovery** treats any redirect as a failure so the credential cannot leak to a redirect target.

Issue developer credentials

Each developer needs their own gateway key to authenticate. Create a credential per developer at the gateway, following your product's credential management documentation.

Confirm a freshly issued key works against the gateway with the same request as **Confirm the gateway routes your models**, replacing `<gateway-key>` with the new `<developer-key>`:

Bash or Zsh

```
curl -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <gateway-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

PowerShell

```
Invoke-RestMethod -Method Post -Uri "https://llm-
gateway.example.com/v1/messages" `
  -Headers @{ "Authorization" = "Bearer <gateway-key>";
"anthropic-version" = "2023-06-01" } `
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
curl -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <developer-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri "https://llm-
gateway.example.com/v1/messages" `
  -Headers @{ "Authorization" = "Bearer <developer-key>";
"anthropic-version" = "2023-06-01" } `
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN="<developer-key>"
```

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "<developer-key>"
```

```
curl -N -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <developer-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 16, "stream":
true, "messages": [{"role": "user", "content": "count to 3"}]}'
```

```
$body = '{"model": "claude-sonnet-4-6", "max_tokens": 16,
"stream": true, "messages": [{"role": "user", "content": "count
to 3"}]}'
$body | curl.exe -N -X POST "https://llm-
gateway.example.com/v1/messages" `
  -H "Authorization: Bearer <developer-key>" `
  -H "anthropic-version: 2023-06-01" `
  -H "content-type: application/json" `
  --data-binary '@-'
```

Checkpoint: a `200` with a `content` field means the developer key reaches the gateway and the gateway forwards it. A `401` here, when **the previous step** succeeded, means the developer key is wrong or hasn't taken effect at the gateway yet.

Issuing one key per developer rather than a shared key is what makes per-developer usage attribution and individual offboarding work. The environment variable that holds the key depends on which header the gateway reads. For a gateway that checks credentials in the `Authorization: Bearer` header, developers set their key in `ANTHROPIC_AUTH_TOKEN`. For a gateway that reads keys from the `x-api-key` header, developers set `ANTHROPIC_API_KEY` instead; the **credential table** covers the mapping.

Test Claude Code against the gateway

Run Claude Code through the gateway yourself before distributing anything, using the same configuration the rollout will deliver fleet-wide. Type these directly in a terminal, not in a `.env` or settings file; they last only for this terminal session, so closing it returns your machine to its normal configuration. Use `ANTHROPIC_API_KEY` instead of `ANTHROPIC_AUTH_TOKEN` if your gateway reads

the `x-api-key` header:

Bash or Zsh

```
curl -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <gateway-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

PowerShell

```
Invoke-RestMethod -Method Post -Uri "https://llm-
gateway.example.com/v1/messages" `
  -Headers @{ "Authorization" = "Bearer <gateway-key>";
"anthropic-version" = "2023-06-01" } `
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
curl -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <developer-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```
Invoke-RestMethod -Method Post -Uri "https://llm-
gateway.example.com/v1/messages" `
  -Headers @{ "Authorization" = "Bearer <developer-key>";
"anthropic-version" = "2023-06-01" } `
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN="<developer-key>"
```

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "<developer-key>"
```

```
curl -N -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <developer-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 16, "stream":
true, "messages": [{"role": "user", "content": "count to 3"}]}'
```

```
$body = '{"model": "claude-sonnet-4-6", "max_tokens": 16,
"stream": true, "messages": [{"role": "user", "content": "count
to 3"}]}'
$body | curl.exe -N -X POST "https://llm-
gateway.example.com/v1/messages" `
  -H "Authorization: Bearer <developer-key>" `
  -H "anthropic-version: 2023-06-01" `
  -H "content-type: application/json" `
  --data-binary '@-'
```

Then send a one-shot prompt through the gateway:

```
claude -p "Reply with one word: connected"
```

Checkpoint: the prompt returns a response, and the request appears in the gateway's log as a `POST` to the `/v1/messages` path with status `200`. Claude Code appends a query string such as `?beta=true`, so match on the path, not the full URL. Two failure messages point in different directions:

- `Not logged in`: check the gateway log to tell the two causes apart. If it's empty, no credential reached the session and no request left the machine; re-run the exports in the shell you're testing from. If it shows a rejected request with `x-api-key` in the `401` body, the gateway expects keys in that header instead; switch to `ANTHROPIC_API_KEY`
- `Failed to authenticate. API Error: 401` means a credential was sent and rejected, and the gateway log says where: a `401` naming `api.anthropic.com` or your provider's endpoint means the gateway reached the upstream but its provider credential was rejected, so the developer key worked and the provider credential the gateway holds is wrong or a placeholder

A wrong or unreachable base URL produces a different symptom: Claude Code **retries the connection with backoff** and can sit with no output for several minutes before reporting an error. If the command appears to hang, check the gateway log instead of waiting; no arriving request means `ANTHROPIC_BASE_URL` doesn't point at the gateway.

Distribute the configuration

Every developer machine needs the gateway address and a credential. You can distribute them

centrally through **managed settings**, so developers configure nothing, or hand developers the values to set themselves.

What to distribute

The same set of variables applies whichever path you choose. Most rollouts only need `ANTHROPIC_BASE_URL` and a credential; include the conditional rows when your gateway setup calls for them.

Variable or setting	What it does	Include when
`ANTHROPIC_BASE_URL`	Sends Claude Code's API requests to the gateway instead of `api.anthropic.com`	Always
`apiKeyHelper`, or a credential in `ANTHROPIC_AUTH_TOKEN` or `ANTHROPIC_API_KEY`	Authenticates each request to the gateway. The helper runs a command to fetch the key; the variables hold a static key, sent as `Authorization: Bearer` and `x-api-key` respectively	Always; one of the three
`ANTHROPIC_CUSTOM_HEADERS`	Adds extra HTTP headers to every API request	Your gateway requires a tenant or routing header on every request
`CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY`	Queries the gateway's `/v1/models` at startup and adds the returned names to the `/model` picker	Your gateway serves `/v1/models` and you want developers' pickers populated from it
`CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS`	Stops Claude Code sending pre-release capability headers and body fields	Your gateway forwards to a Bedrock or Vertex upstream that rejects beta fields; see **Gateway requirements**
`ANTHROPIC_MODEL` or `ANTHROPIC_DEFAULT_HAIKU_MODEL`	Set which model name Claude Code requests for the main session and for background traffic	Your gateway routes model names that don't match Claude Code's defaults, or you route **background functionality** to a different model. Route both the override names and Claude Code's default names at the gateway, since some sub-calls can request the default name regardless of the override
`ANTHROPIC_BEDROCK_BASE_URL`, `ANTHROPIC_VERTEX_BASE_URL`, `ANTHROPIC_FOUNDRY_BASE_URL`, or `ANTHROPIC_AWS_BASE_URL` with the **variables for that provider**	Point Claude Code at the gateway through a provider-specific base URL. Bedrock and Vertex also switch to those providers' native request format	Your gateway fronts Bedrock, Vertex, Foundry, or the Claude Platform on AWS; see **API formats**

Distribute through managed settings

Deliver the variables through the `env` block of a **managed settings file**, pushed by MDM, registry policy, or configuration management:

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://llm-gateway.example.com"
  },
  "apiKeyHelper": "/usr/local/bin/get-gateway-key"
}
```

Add the conditional variables from the table to the same `env` block. A managed `ANTHROPIC_BASE_URL` is enforced and cannot be overridden by a developer's shell export, since Claude Code applies it over the process environment and lower-precedence settings.

Do not include `forceLoginMethod` or `forceLoginOrgUUID` in managed settings alongside a gateway credential. On Claude Code v2.1.146 and later, either key blocks `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, and `apiKeyHelper` at startup, so developers see `This machine's managed settings require a first-party login` and cannot proceed.

Server-managed settings delivery requires a direct connection to `api.anthropic.com`, so it does not reach gateway-routed sessions. Gateway deployments use this file-based managed settings path, which enforces the same keys.

For the credential, distribute one `apiKeyHelper` command in the managed settings file as shown above; the command authenticates to your secrets store as the local developer, so each machine receives its own key. Alternatively, deliver each developer their key through your existing secrets process and have them set `ANTHROPIC_AUTH_TOKEN` themselves.

Some environments need separate delivery:

- The desktop app reads gateway routing only from its MDM-delivered third-party inference configuration; deploy that file alongside managed settings so desktop sessions route through the gateway too. See the **desktop third-party configuration docs** and the **desktop gateway docs**
- CI runners need `ANTHROPIC_BASE_URL` and the credential set in the **runner's environment**
- WSL on managed Windows machines reads the Windows managed settings only when `wslInheritsWindowsSettings` is `true`

Hand developers the values to set themselves

If you don't have managed-settings distribution in place, send each developer what they need to follow the **connect page**:

- The gateway URL
- Their personal credential
- **Which variable to put the credential in**: `ANTHROPIC_AUTH_TOKEN` for a bearer-token gateway, or `ANTHROPIC_API_KEY` for an `x-api-key` gateway. Telling developers which one saves them the trial-and-error described on the **connect page**
- Any conditional variables from the **What to distribute table**, with their values

The **connect page** walks developers through setting each one.

Checkpoint: on a developer machine, `claude` starts a session without showing the login screen, since the distributed credential satisfies authentication. Then run `/status` and open the **Status** tab: the `Anthropic base URL` line shows the gateway address, and for managed distribution the `Setting sources` line includes managed settings. A login screen, or a missing `Anthropic base URL` line, means the configuration didn't reach the machine.

Verify the rollout

Confirm everything works from a developer machine, not the gateway host, so the test covers the network path developers use. Send a streaming request, which checks the endpoint, streaming pass-through, and model routing at once:

Bash or Zsh

```
curl -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <gateway-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

PowerShell

```powershell
Invoke-RestMethod -Method Post -Uri "https://llm-
gateway.example.com/v1/messages" `
  -Headers @{ "Authorization" = "Bearer <gateway-key>";
"anthropic-version" = "2023-06-01" } `
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```bash
curl -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <developer-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 1, "messages":
[{"role": "user", "content": "."}]}'
```

```powershell
Invoke-RestMethod -Method Post -Uri "https://llm-
gateway.example.com/v1/messages" `
  -Headers @{ "Authorization" = "Bearer <developer-key>";
"anthropic-version" = "2023-06-01" } `
  -ContentType "application/json" `
  -Body '{"model": "claude-sonnet-4-6", "max_tokens": 1,
"messages": [{"role": "user", "content": "."}]}'
```

```bash
export ANTHROPIC_BASE_URL=https://llm-gateway.example.com
export ANTHROPIC_AUTH_TOKEN="<developer-key>"
```

```
$env:ANTHROPIC_BASE_URL = "https://llm-gateway.example.com"
$env:ANTHROPIC_AUTH_TOKEN = "<developer-key>"
```

```
curl -N -X POST "https://llm-gateway.example.com/v1/messages" \
  -H "Authorization: Bearer <developer-key>" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-6", "max_tokens": 16, "stream":
true, "messages": [{"role": "user", "content": "count to 3"}]}'
```

```
$body = '{"model": "claude-sonnet-4-6", "max_tokens": 16,
"stream": true, "messages": [{"role": "user", "content": "count
to 3"}]}'
$body | curl.exe -N -X POST "https://llm-
gateway.example.com/v1/messages" `
  -H "Authorization: Bearer <developer-key>" `
  -H "anthropic-version: 2023-06-01" `
  -H "content-type: application/json" `
  --data-binary '@-'
```

You should see `data:` lines arrive incrementally. The whole response arriving at once after a pause means the gateway is buffering, which stalls Claude Code; a `404` means the model name isn't routed. Repeat per model name.

Then start `claude` and send a message. Each symptom at this step has one cause:

- A login prompt means a credential gap. Run `/status` and open the **Status** tab: when the `Setting sources` line doesn't include managed settings, the distribution didn't reach the

machine; when it does, the developer credential wasn't delivered, so set `ANTHROPIC_AUTH_TOKEN` or the `apiKeyHelper`

- `Failed to authenticate` errors mean the gateway is rejecting requests; its log says which credential failed. A rejection the gateway logs itself names the developer key, while a `401` from `api.anthropic.com` or your provider's endpoint means the provider credential the gateway holds was rejected
- A one-time approval prompt for the key is expected on first use when the gateway expects keys in the `x-api-key` header, set as `ANTHROPIC_API_KEY`. With `ANTHROPIC_AUTH_TOKEN`, no prompt appears and the variable takes over silently; a previously saved claude.ai login is inactive for that session

Finally, check the gateway's logs for the message you sent: the credential identifies the developer, and the `x-claude-code-session-id` **header** groups requests by session. If features fail with the **troubleshooting symptoms**, the gateway is stripping headers or rewriting errors; see the **gateway requirements** above.

Maintain the gateway

After rollout, three kinds of change reach the gateway over time. Each has a symptom to watch for and an action to take.

Change	Symptom when the gateway hasn't kept up	Action
New Claude Code releases add `anthropic-beta` values and request body fields	Developers report `400` errors naming a new field after they update Claude Code; see **feature pass-through**	Forward `anthropic-*` headers and request bodies verbatim rather than allowlisting; test new Claude Code releases against the gateway before they reach developers
New Claude models become available	Developers selecting a new model name get `404`; the `/model` picker doesn't list it	Add the model name to the gateway's routing configuration, then re-run the **routing check**. If you distribute `ANTHROPIC_MODEL` or the default-model variables, update the managed settings
Credentials expire or need rotation	All developer requests start failing with `401` from the upstream	Rotate the gateway's provider credential on its own schedule; developer keys rotate at the gateway, and an `apiKeyHelper` handles per-developer rotation without redistributing settings

When sizing per-key rate limits, account for the client **retrying transient failures**, including `429`

responses, up to 10 times with backoff, honoring `Retry-After`. Keep the **protocol reference** as the contract for what each Claude Code release sends.

Related resources

- **Connect Claude Code to an LLM gateway**: the developer-facing setup steps, with per-surface configuration and a troubleshooting table you can hand to developers
- **Gateway protocol reference**: the wire contract for gateway operators, covering endpoints, headers to forward, and the feature pass-through table
- **Settings files and precedence**: how managed, project, and user settings combine, and where the managed file goes on each platform
- **Set up Claude Code for your organization**: the wider rollout this gateway is one part of, including policy enforcement, usage visibility, and data handling